

# **Compiladores e Intérpretes**

## **Pautas Etapa 4**

### **Analizador Semántico Parte 2**

**Universidad Nacional del Sur**  
**Departamento de Ciencias e Ingeniería de la Computación**  
**2025**



# Introducción - Aclaraciones

- El desarrollo de esta **etapa** del proyecto se **corresponde** con los **Tema 4.1, 4.2, 4.5.1, 4.5.2, 4.5.3 y 4.6** de la materia
- Te recomendamos que **hayas mirado y tengas frescos** todos los temas allí tratados **antes de arrancar** el diseño/desarrollo de esta etapa
- También te **esencial** mirar el documento de **MiniJava-Semantica Completo**
- En los ejemplos de los diseños de esta presentación vas a ver que hay "...", eso quiere decir que es muy posible que a el diseño tenga algunas/varias cosas mas

# Introducción

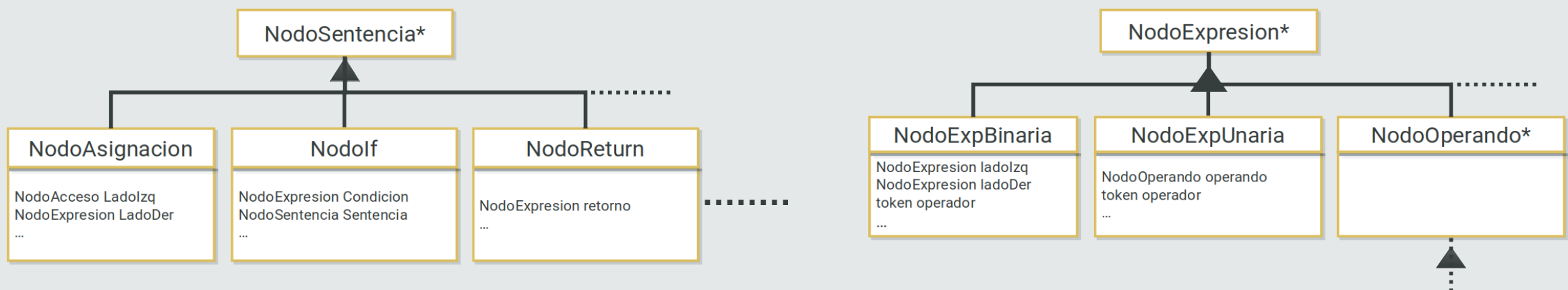
- En esta **etapa** hay que **diseñar e implementar** un **el Chequeo de Sentencias del Analizador Semántico** para **MiniJava**
- Para esto habrá que extender al **Analizador Sintáctico** de la etapa anterior para que cree que los **ASTs** de los métodos/constructores y los asocie a sus entradas en la Tabla de Símbolos (**TS**)
- El **Chequeo de Sentencias** se realizará directamente sobre esos **ASTs** en la **TS**
- Además, de manera similar a la etapa anterior va ser necesario un **Modulo Principal** que:
  - Manejará la **interfaz** con el **usuario** y
  - **Reportará los errores** detectados por el Semántico, el Sintáctico y el Léxico
  - En caso de no haber errores indicará que el análisis fue exitoso



# Diseño de los ASTs y el Chequeo de Sentencias

# Diseño de la Estructura de los ASTs

- Para diseñar la ASTs te recomendamos mirar 4.5.1 y 4.5.2
- Primero hay que **identificar y clasificar** los **tipos de nodos** del AST
  - La idea general es que **cada tipo de sentencia y expresión** se corresponderá con un **tipo de nodo** (el cual a su vez será una **clase**)
  - En las **clases** los **atributos** se usan para representar la **información** y los **hijos** que el tipo de nodo puede tener
  - Es importante también identificar la relación “**es-un**” (modelado con herencia) entre los tipos de nodos (e.g. una Expresion Binaria es una Expresión, o un Literal es un Operando)



# Diseño de la Construcción de los ASTs

- Los **ASTs** se **construyen** a medida que se realiza el **análisis sintáctico**
  - Para **diseñar** que hacer usamos una **EDT**
    - Esta EDT debe realizarse sobre la **versión de la gramática** de MiniJava que usaron para **implementar el analizador sintáctico**
    - La idea es que las **acciones** de la EDT vayan **construyendo los ASTs** a medida que vamos **reconociendo sintacticamente** cada sentencia
    - Una vez **armado el AST del bloque** de un **método/constructor** se asocia a la entrada del subprograma en la **TS**
  - Para esto, además de **4.5.3**, te recomendamos que mires **4.2** y **4.2.1** para entender las **EDTs**

# Diseño de la Construcción de los ASTs

- Los **ASTs** se **construyen** a medida que se realiza el **análisis sintáctico**

- Para **diseñar** que hacer usamos una **EDT**
  - Esta EDT debe realizarse sobre la **versión** que usaron para **implementar** el **analizador** sintáctico
  - La idea es que las **acciones** de la EDT van cambiando a medida que vamos **reconociendo sintáctico**
  - Una vez **armado el AST del bloque** de un **método/constructor** se asocia a la entrada del subprograma en la **TS**
- Para esto, además de **4.5.3**, te recomendamos que mires **4.2** y **4.2.1** para entender las **EDTs**

Con la **EDT terminada**, como pueden ver en 4.5.3, tenemos el **diseño** para **modificar el analizador sintáctico** y que vaya creando los ASTs de los bloques que aparecen en los métodos y constructores del programa

# Diseño del Chequeo de Sentencias

- Para el Chequeo de Sentencias te recomendamos mirar el tema **4.6**
- Consiste de **dos tareas** principales que se realizan en paralelo
  - **Resolución de Nombres**
  - Control de la **correctitud** (incluyendo el **chequeo de tipos**) de todos los nodos de cada AST
- Estas tareas se pueden **diseñar** como funciones/**métodos** directamente sobre la **TS** y las clases de las **Nodos**

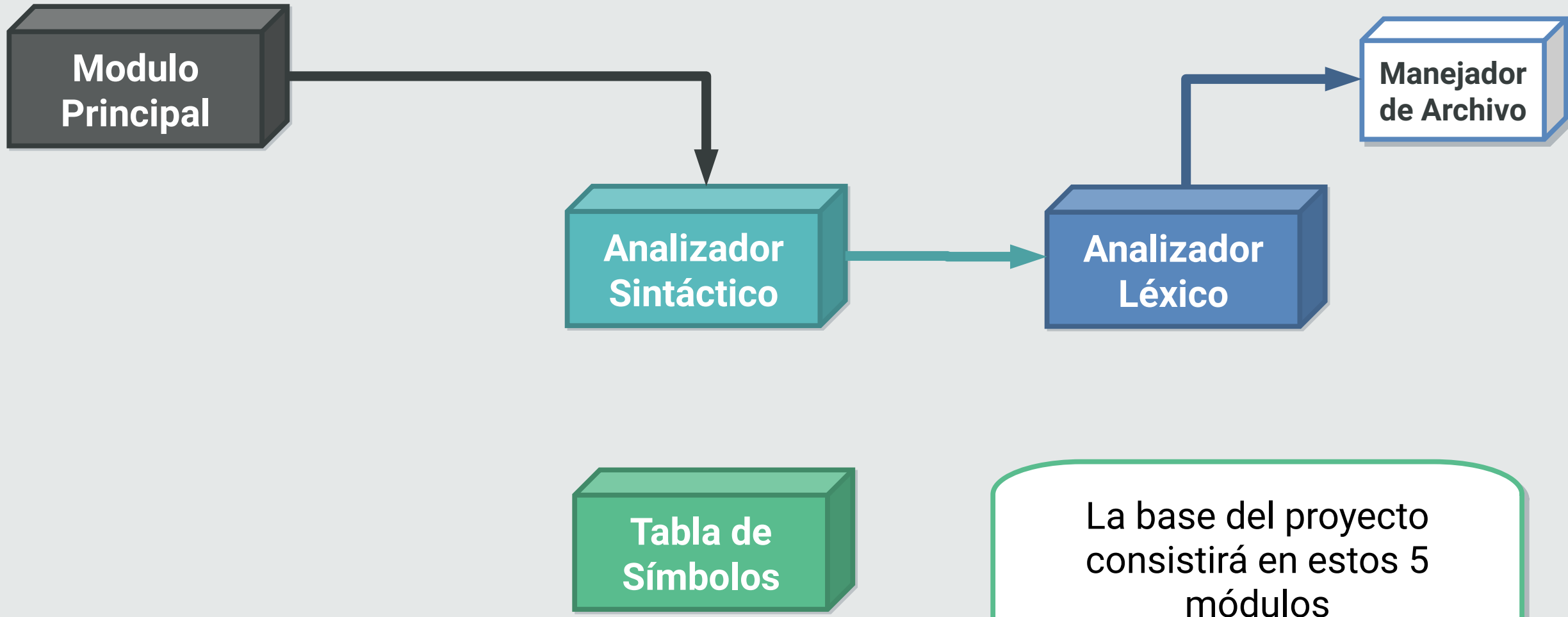


# Diseño del Chequeo de Sentencias

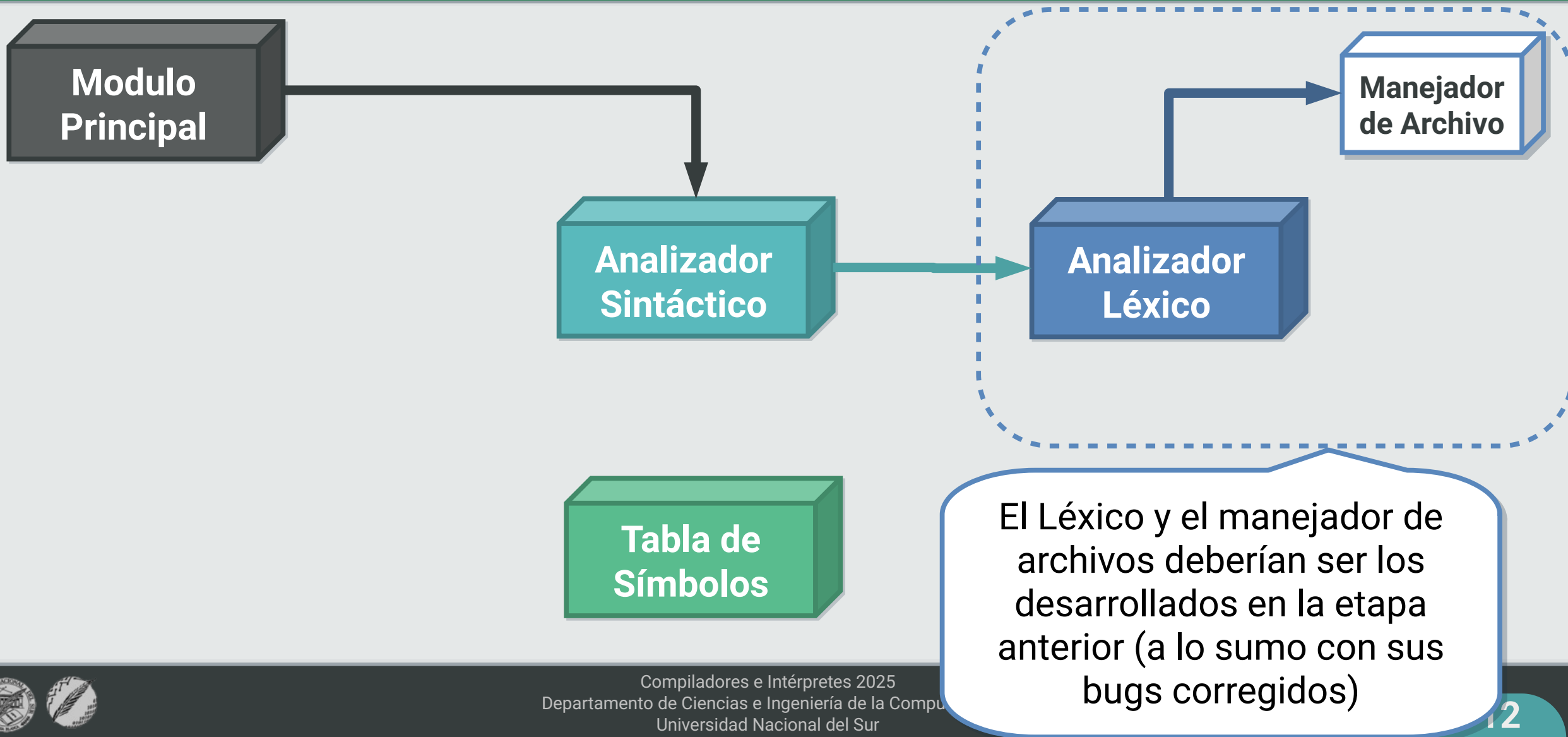
- Para diseñar esta tarea la clave es valernos de la **Delegación** y las **Responsabilidades** de cada entidad
  - La idea es que cada nodo **controla** lo que le **corresponde** (su **responsabilidad**) y **solicita** a cada hijo que se controle (**delega** el control)

# Estructura y Funcionalidad del Software a Desarrollar

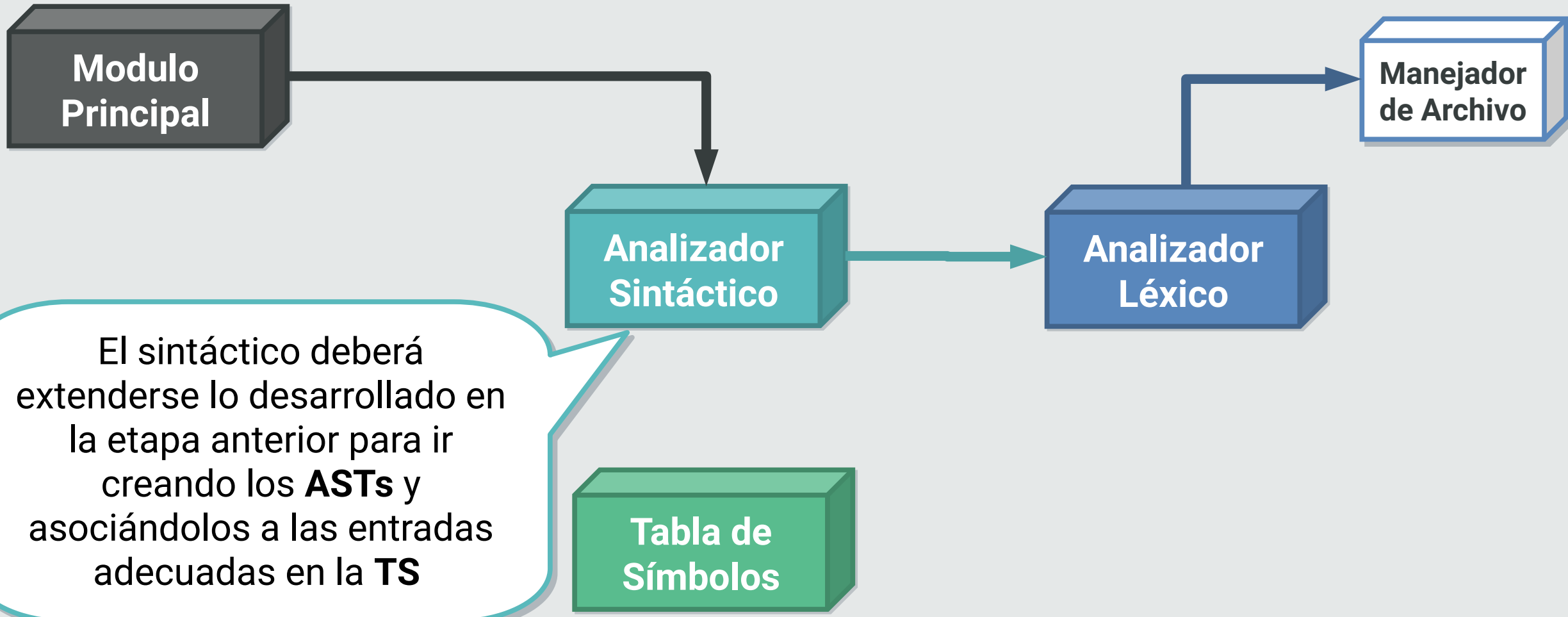
# Esquema General



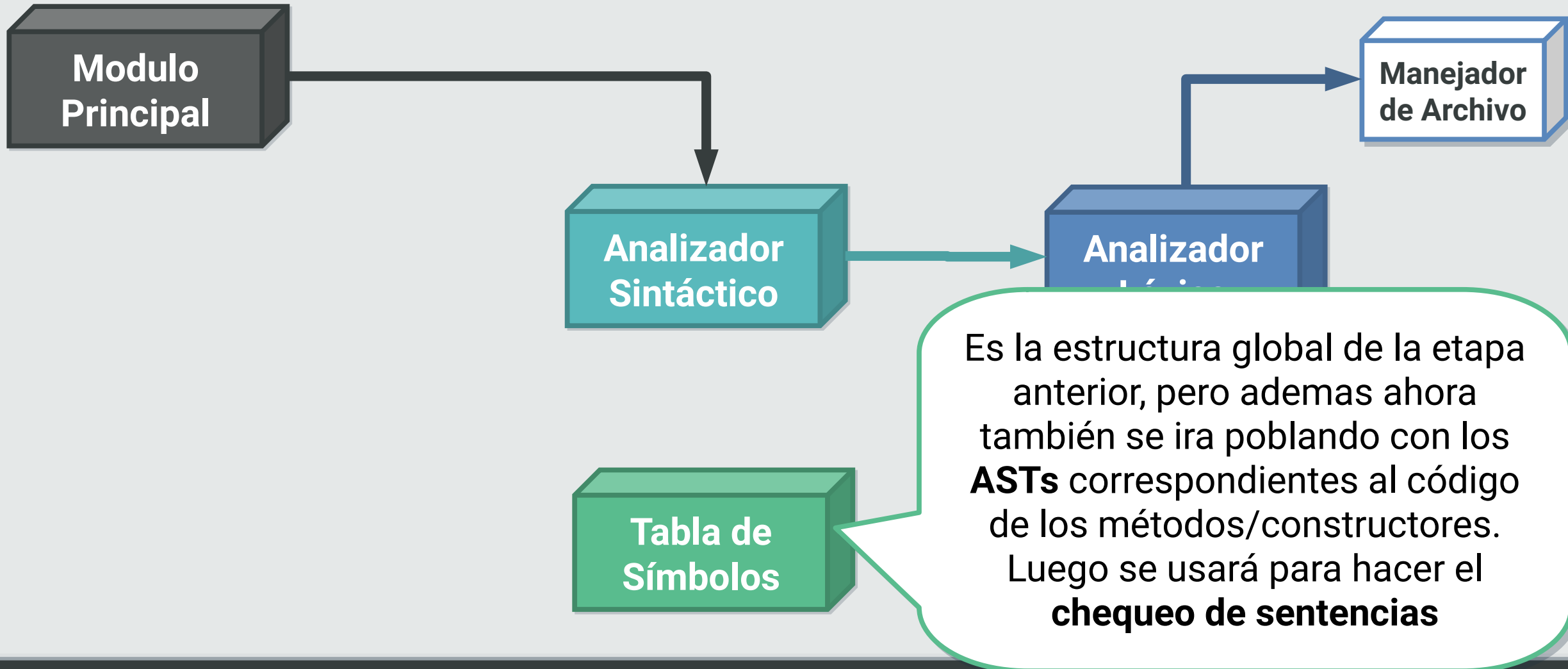
# Esquema General



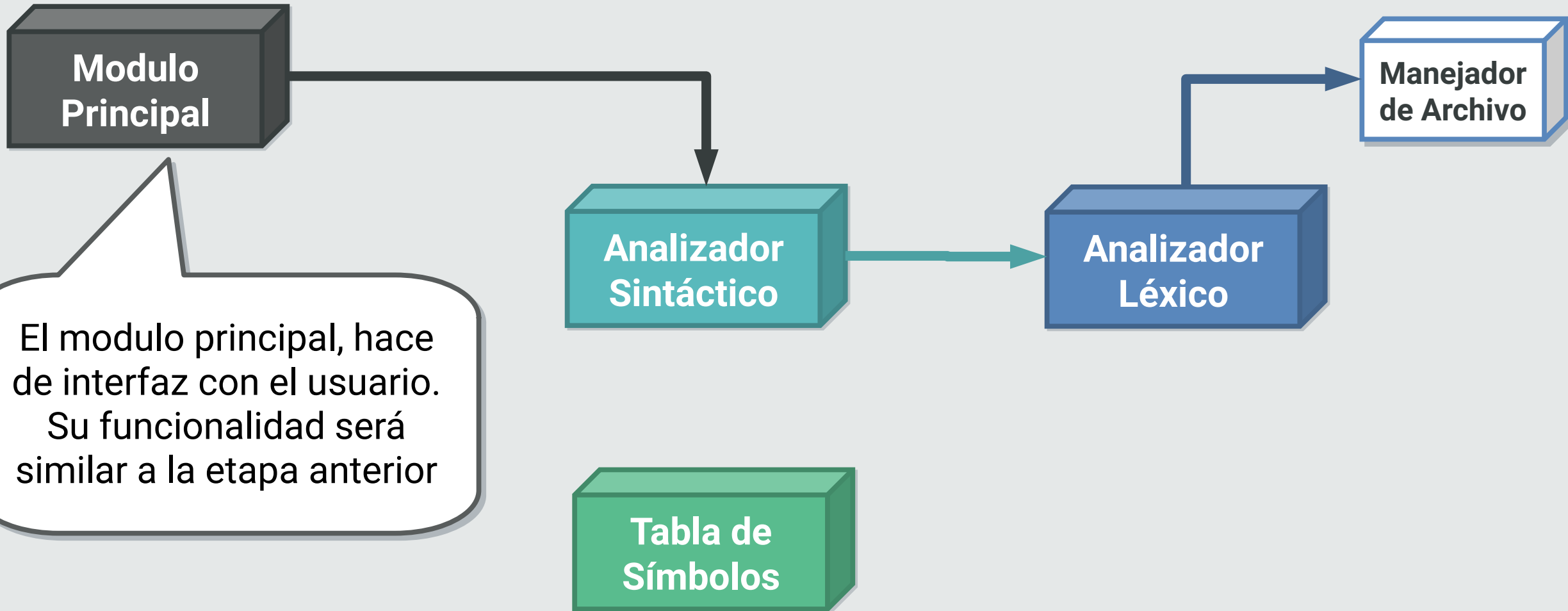
# Esquema General



# Esquema General



# Esquema General



# Esquema General

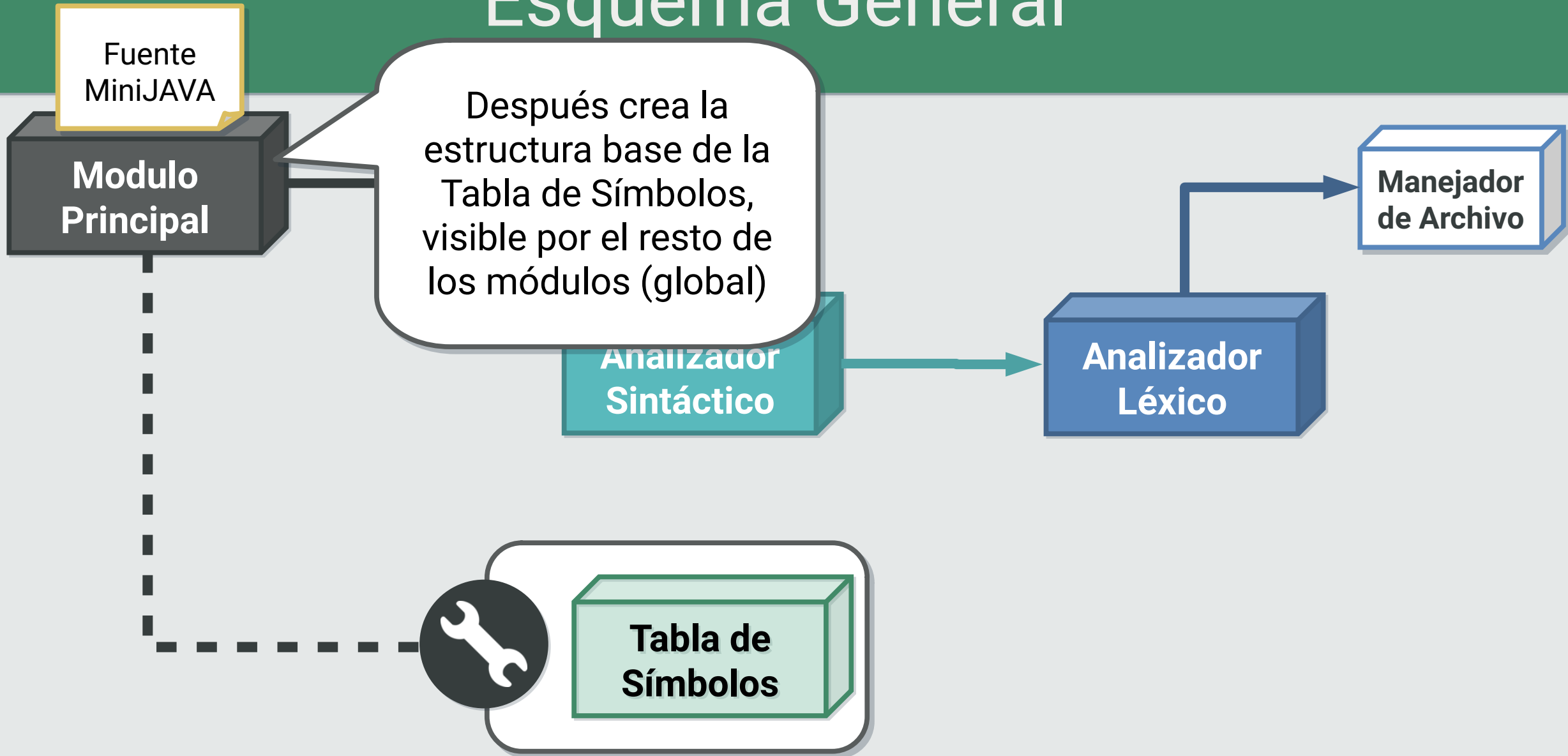




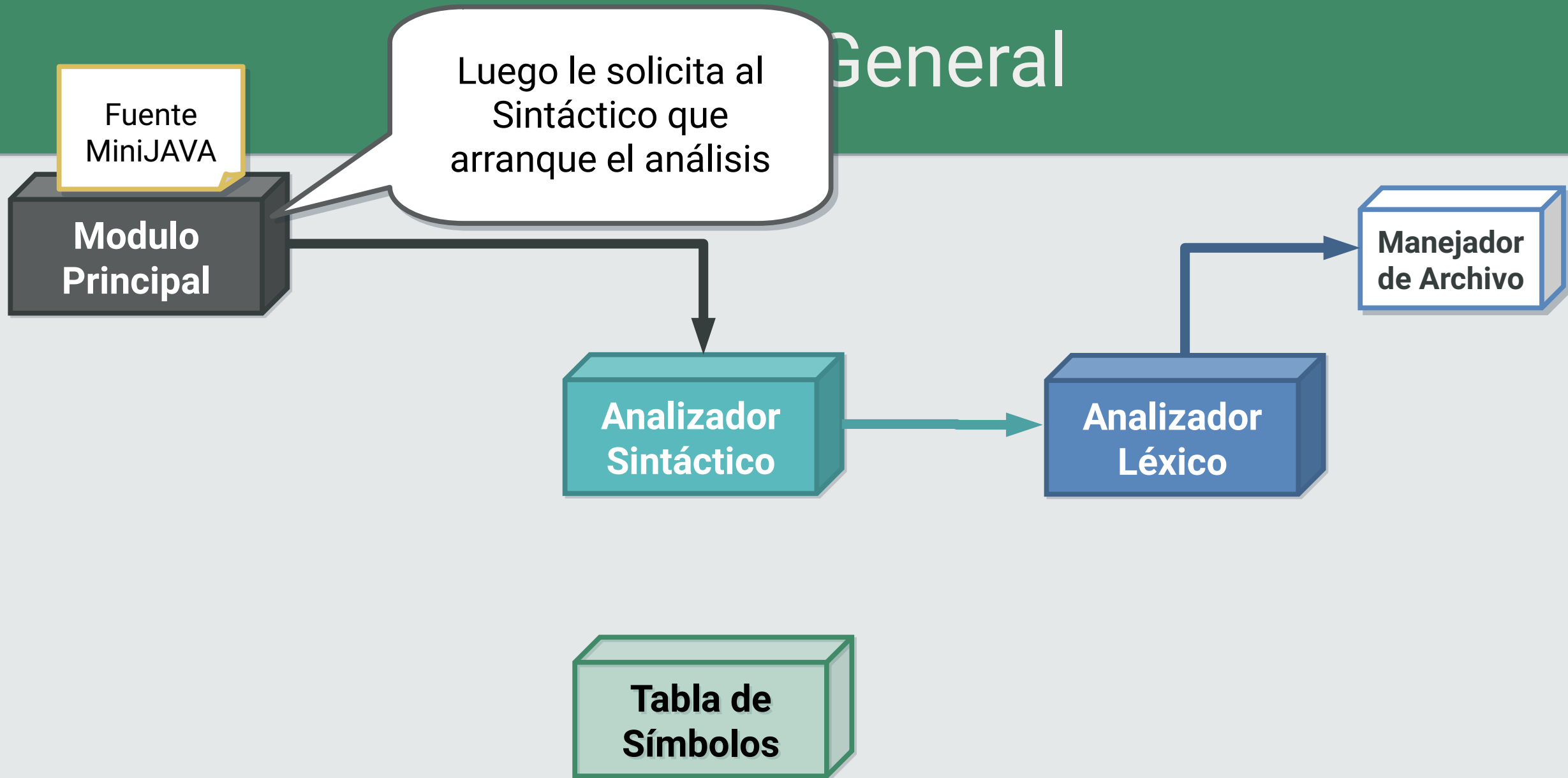
# Esquema General



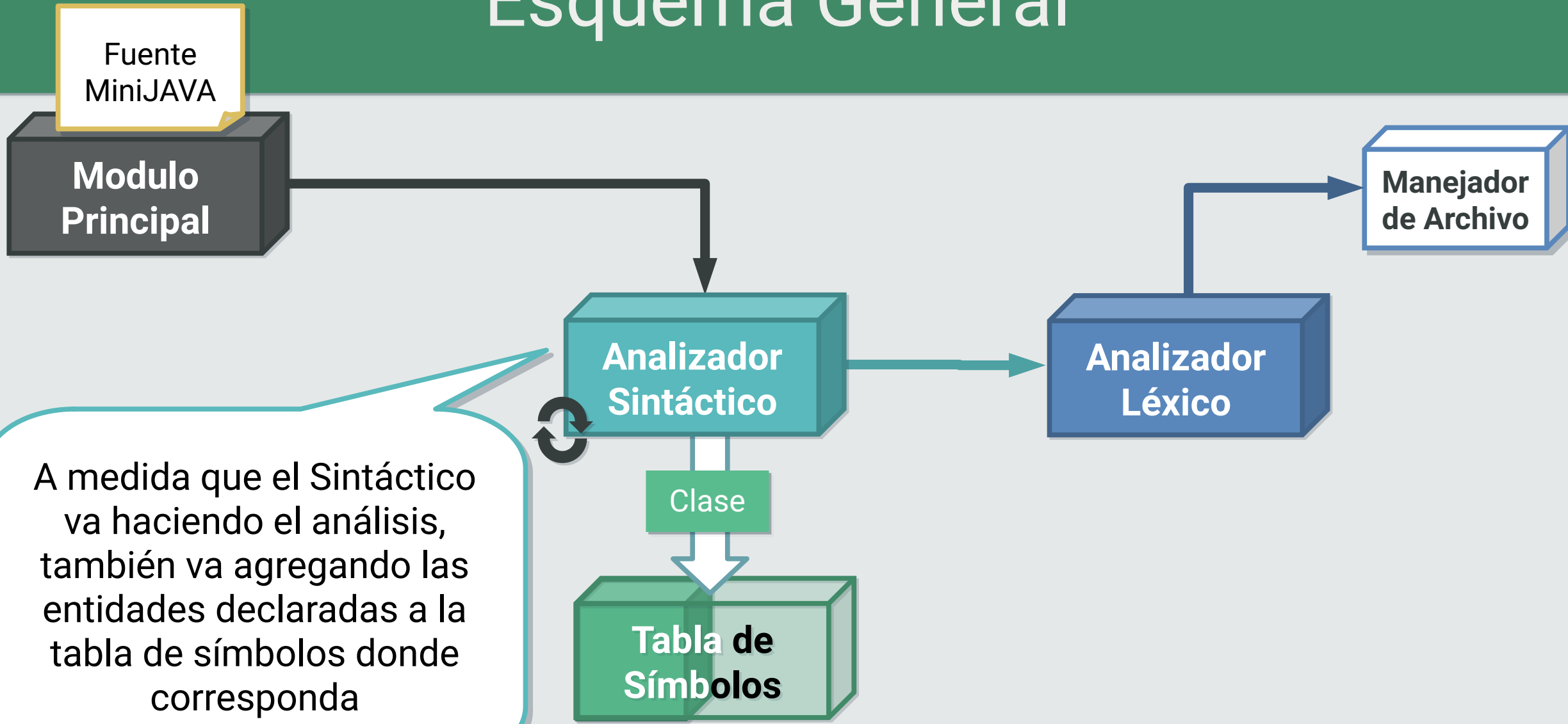
# Esquema General



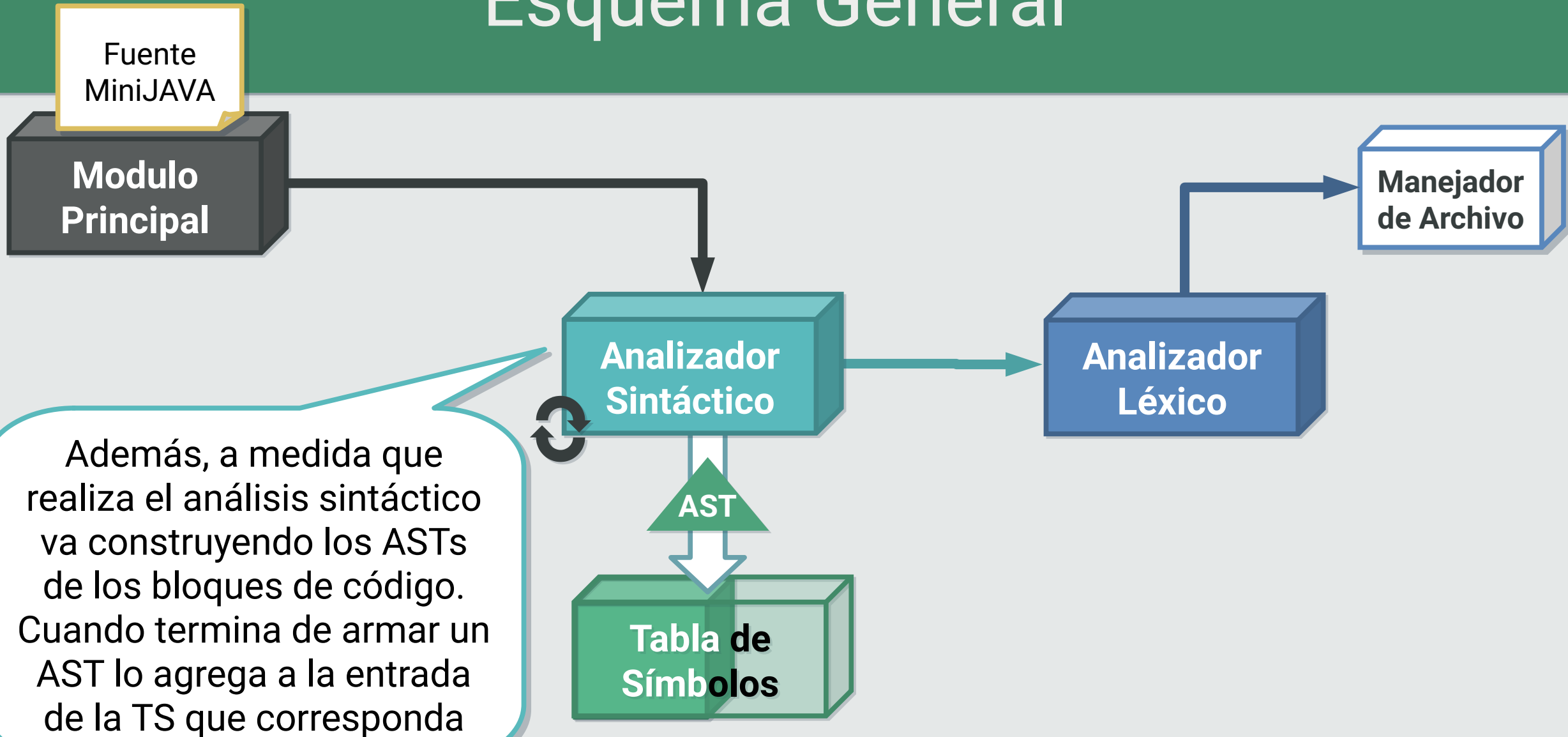
# General

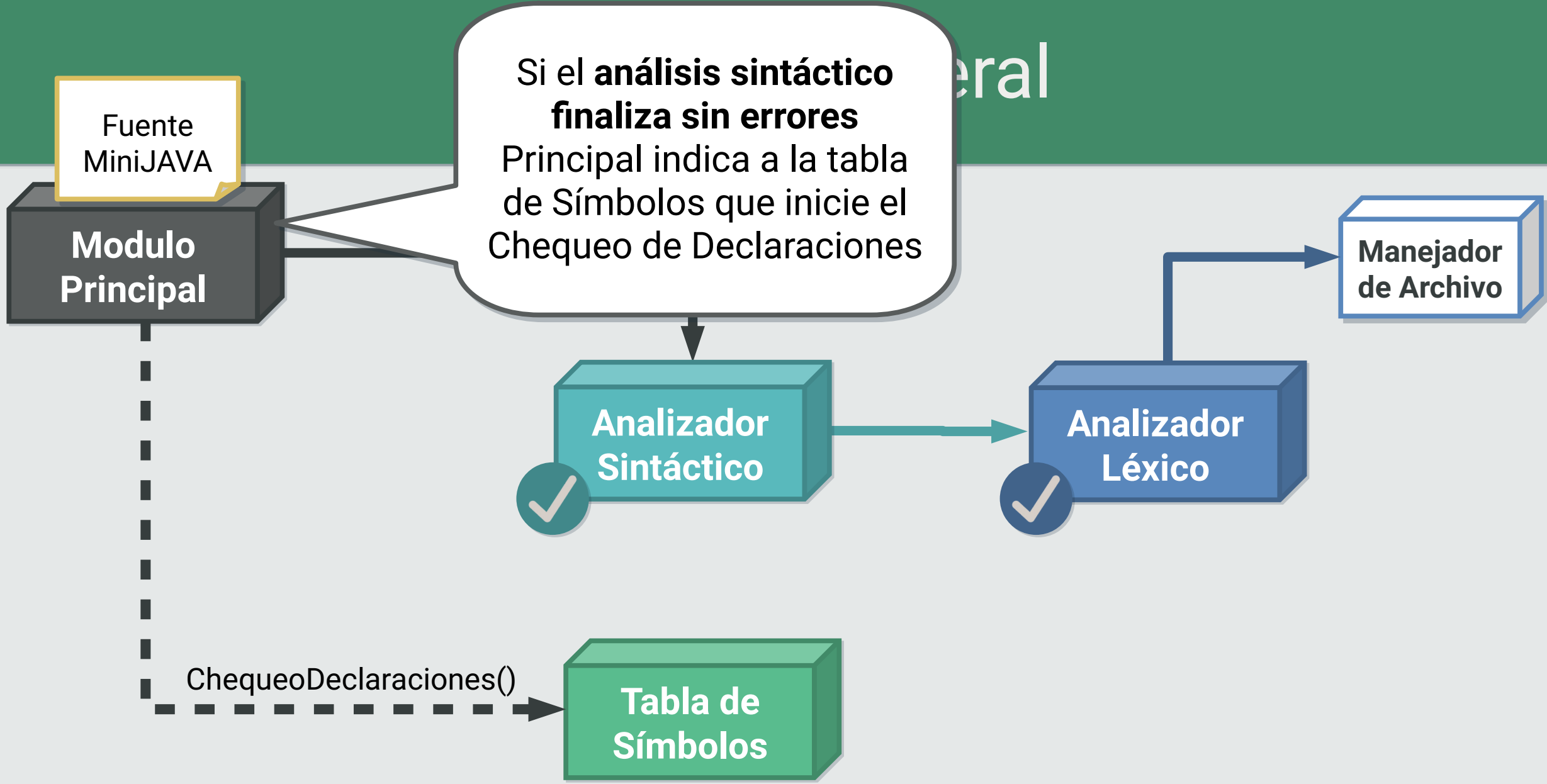


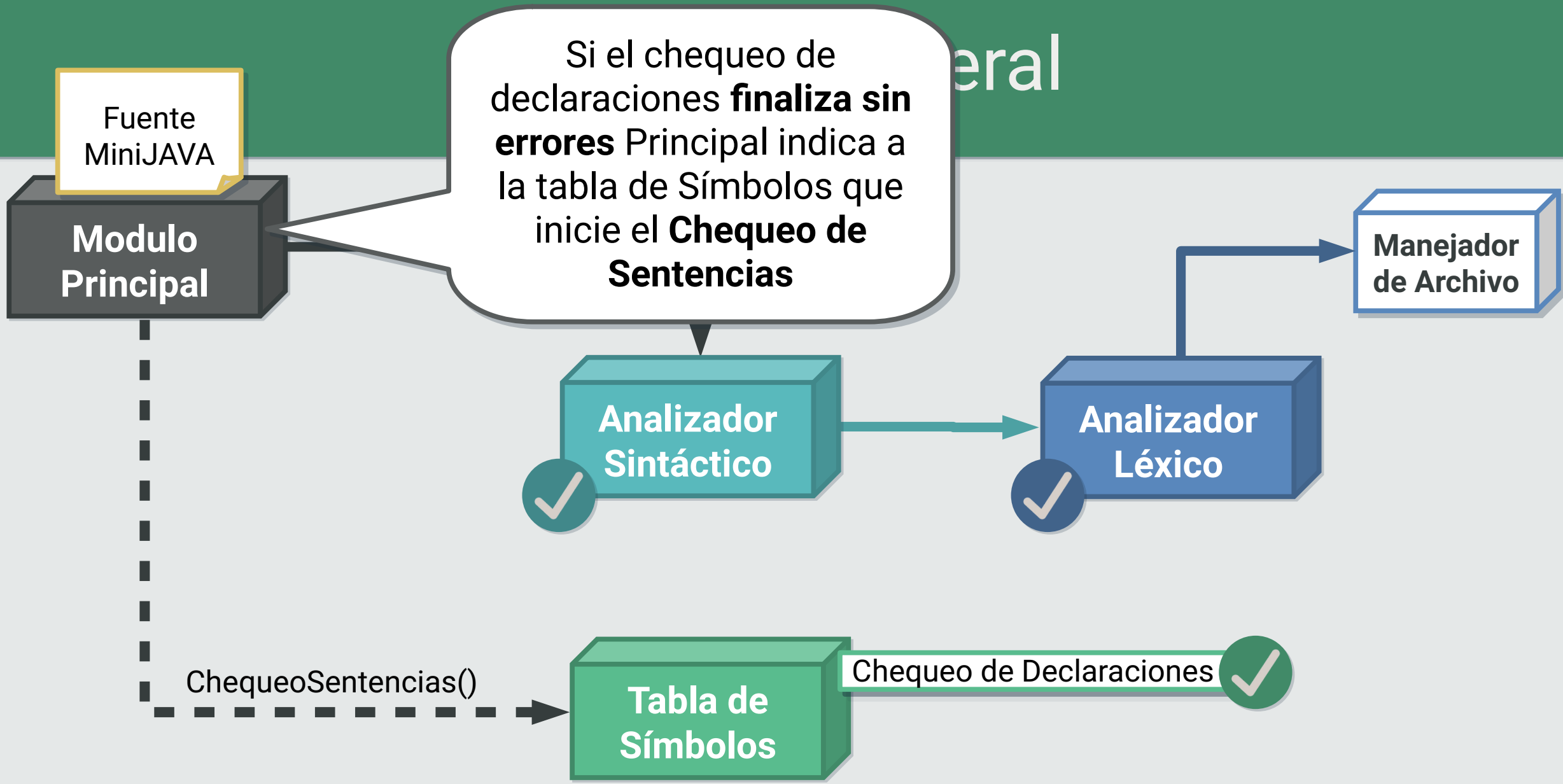
# Esquema General



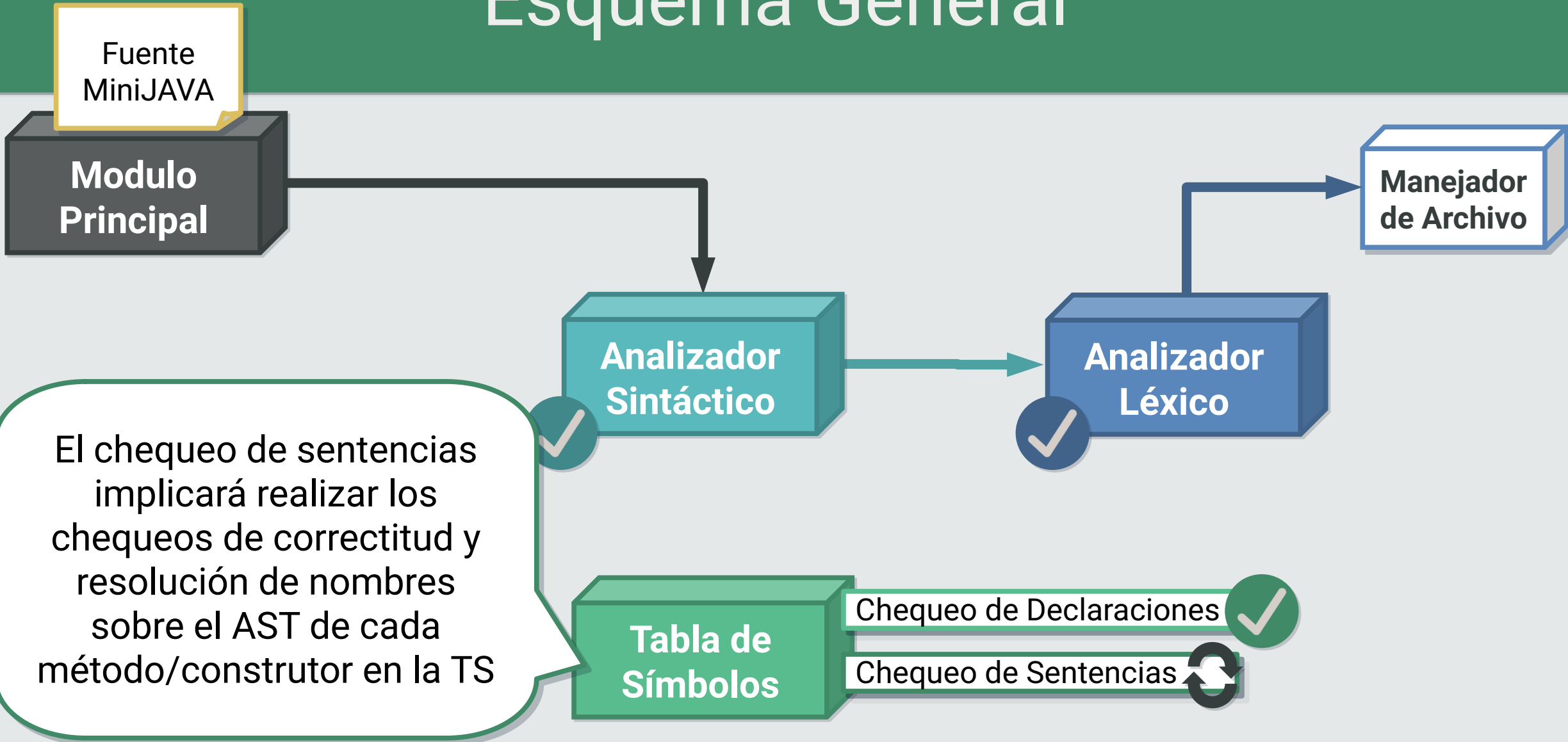
# Esquema General



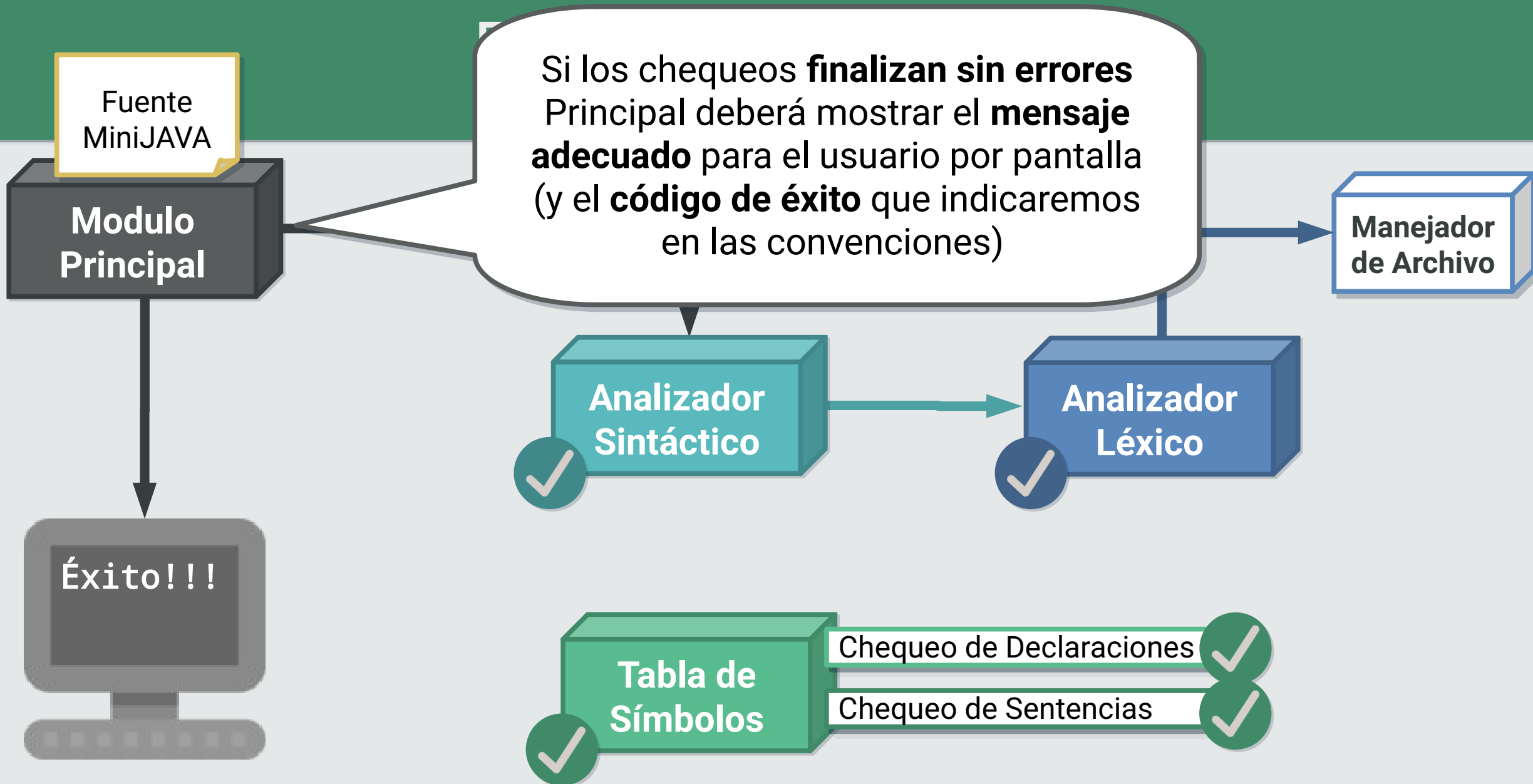


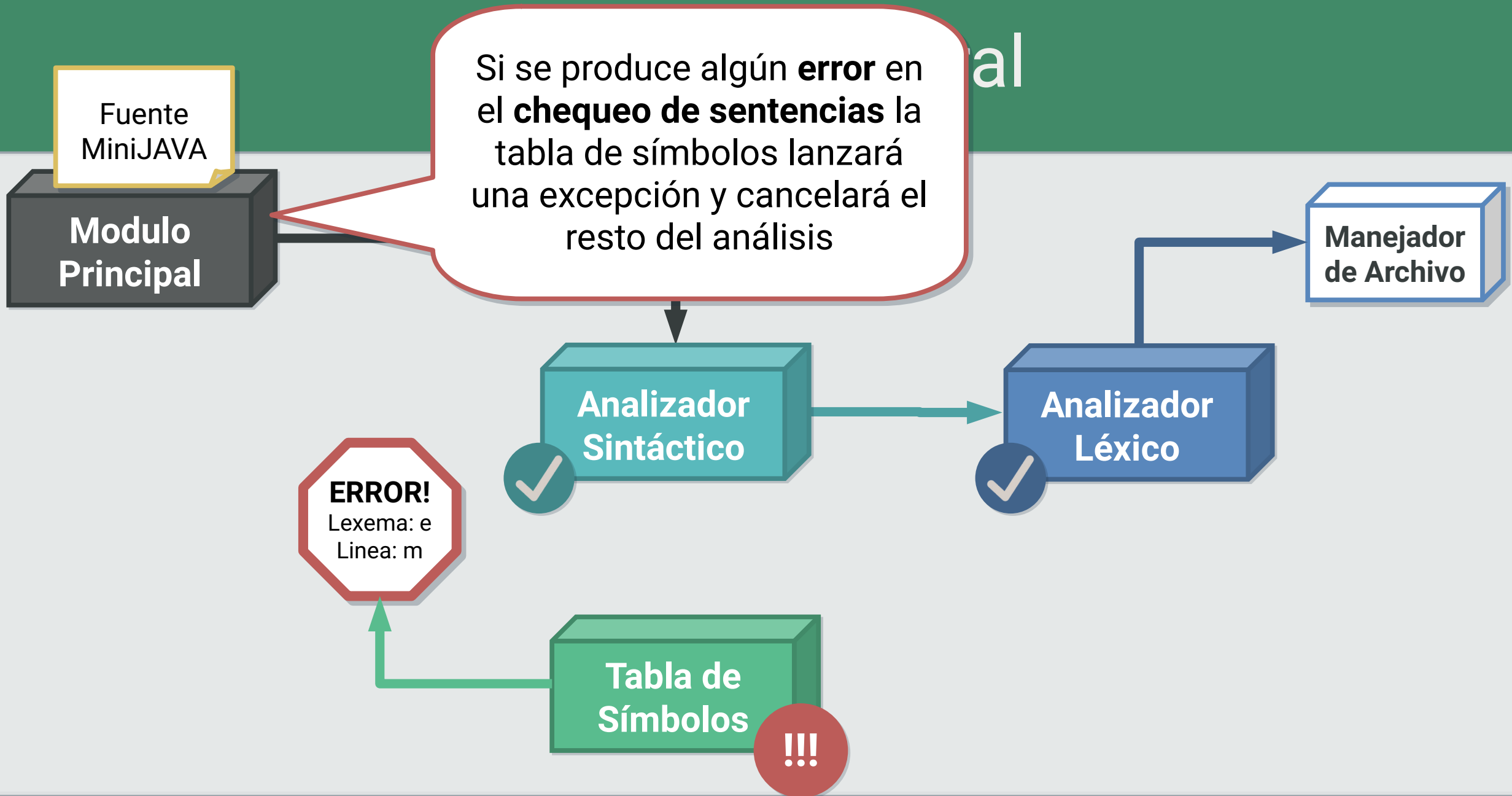


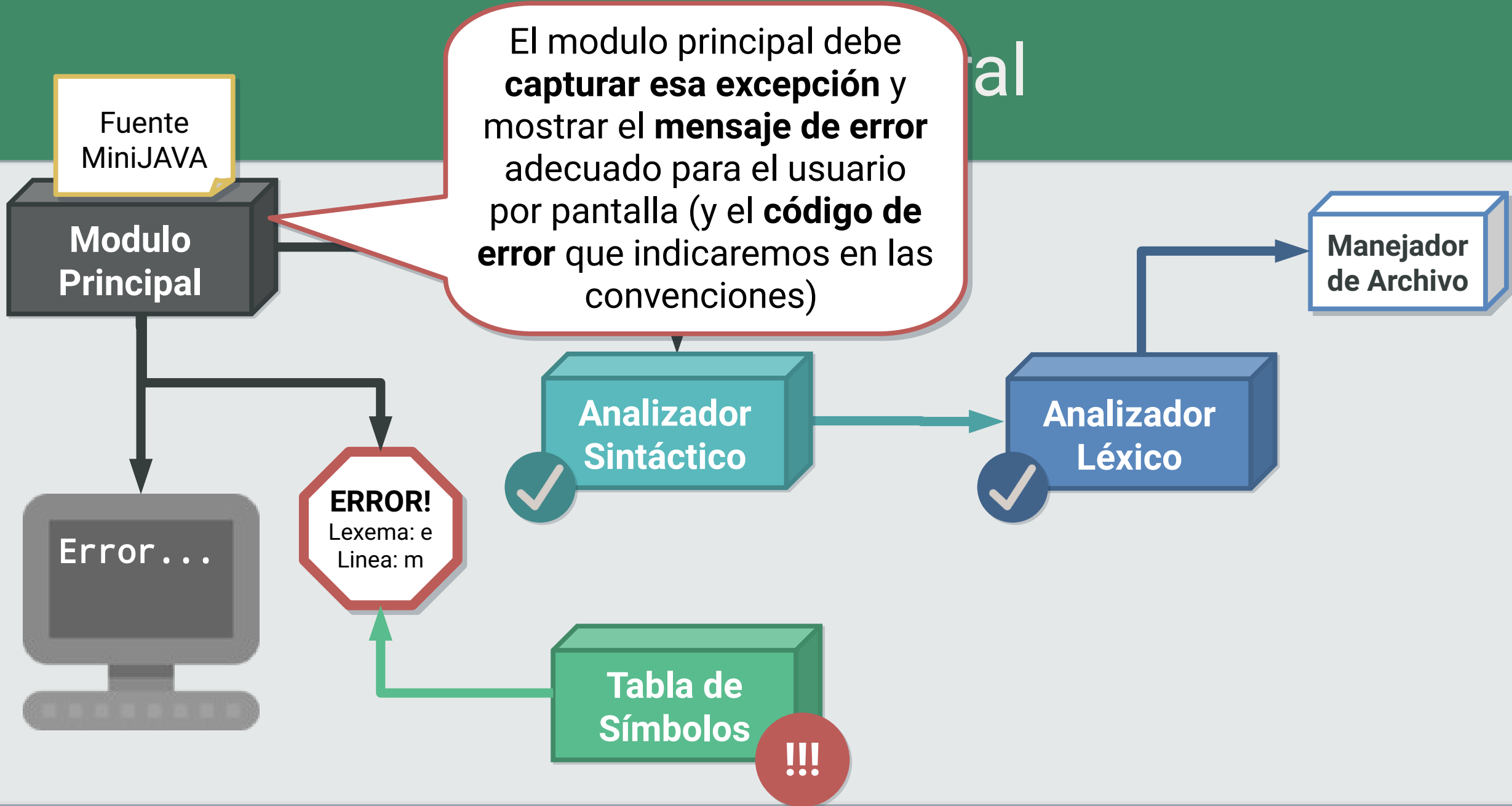
# Esquema General

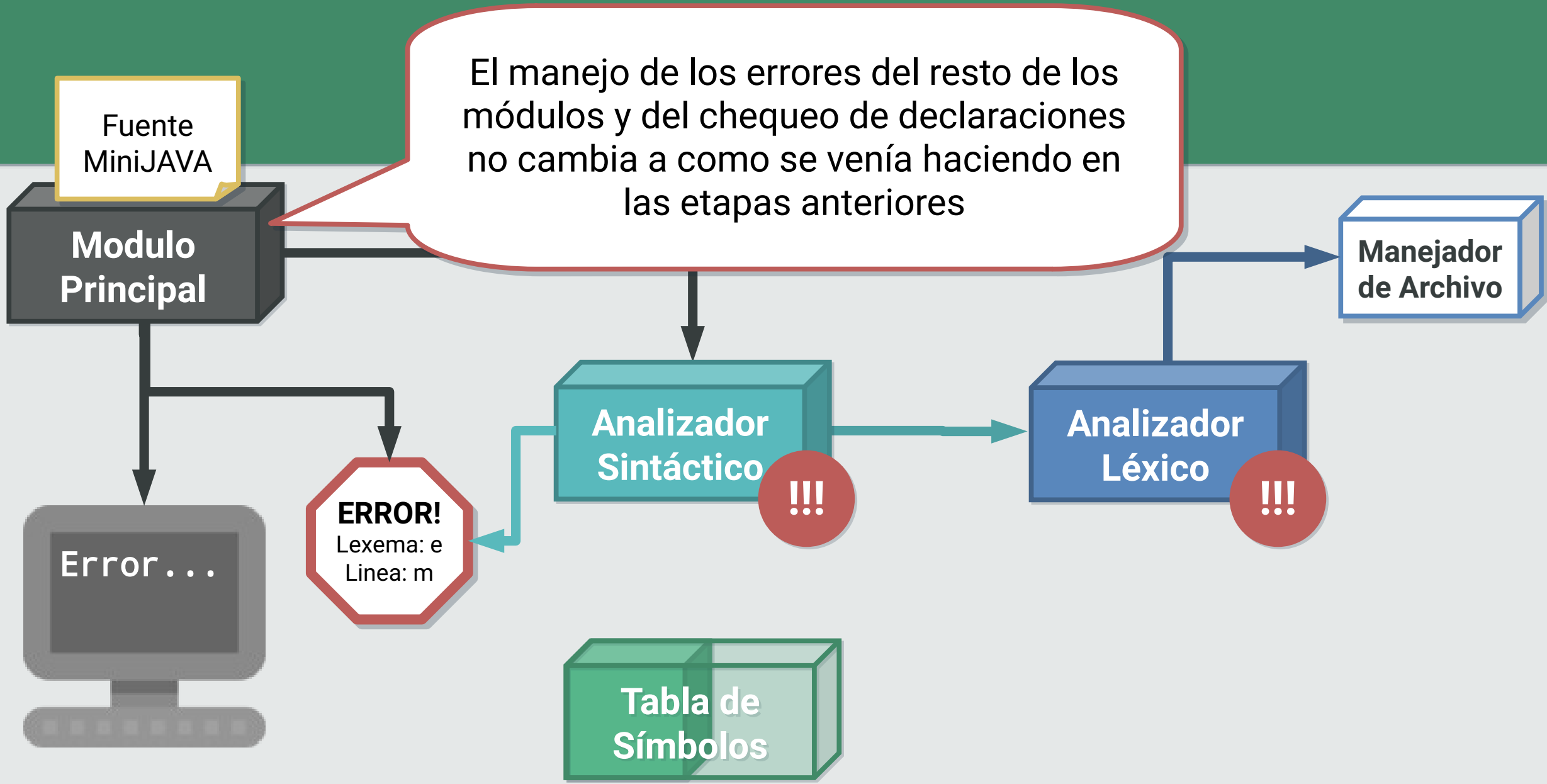












# Convenciones y Consideraciones

# Convenciones y Consideraciones

**Fecha Limite** de Entrega: 31/10/2025 a las 16hs

**Forma de Entrega:** En la actividad asociada a la entrega etapa 4 en Moodle, subiendo un zip con todo lo requerido

# Convenciones y Consideraciones

La **interacción** de la aplicación entregada con el **usuario** deberá ser por **consola**



La entrega deberá consistir de: los archivos el **código fuente** (solo archivos **".java"**), un catálogo con los **casos de test** utilizados para la prueba del software y el **informe en formato pdf**



La **implementación** de las **ASTs** y el **Chequeo de Sentencias** deberá corresponderse con los esquemas presentados en los Temas 4.5.1, 4.5.2, 4.5.3 y 4.6



# Convenciones y Consideraciones

## Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token correspondiente a la nodo que tiene el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Ejemplo:



*Fuente MiniJava*

```
class A{  
    void m1(){  
        (4+true);  
    }  
}
```

El Reporte de  
Error debería ser

```
Error Semántico en linea 3: El tipo booleano  
es incompatibles con el operador "+"  
[Error: + | 3]
```



# Convenciones y Consideraciones

## Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token correspondiente a la nodo que tiene el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Otro Ejemplo:



*Fuente MiniJava*

```
class A{  
    void m1(){  
        a = 10;  
    }  
}
```

El Reporte de  
Error debería ser

```
Error Semántico en linea 3: La variable a no  
fue declarada  
[Error:a|3]
```

# Convenciones y Consideraciones

## Mensaje de Análisis Exitoso

- Cuando no se detectan errores en el análisis el Modulo Principal debería mostrar un mensaje adecuado y luego el código de éxito: **[SinErrores]**
- Por ejemplo

*Fuente MiniJava*

```
class C1{  
}
```

El mensaje  
debería ser

Compilacion Exitosa  
[SinErrores]



# Convenciones y Consideraciones

## Invocación del Compilador

- Al ser compilado el software entregado poder invocarse de consola recibiendo como parámetro el archivo fuente MiniJava (debe aceptar cualquier tipo de extensión!)
- Por ejemplo: si al compilarlo lo llamamos Compilador y el fuente MiniJava se llama programa1.java, debería poder invocarse de la siguiente manera

```
java -jar Compilador.jar programa1.java
```



# Convenciones y Consideraciones



**No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega**

# Convenciones y Consideraciones



**No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega**

Las convenciones son esenciales para que **nosotros podamos corregir** bien la entrega...  
por eso somos tan estrictos!

Si no la podemos corregir  
va a estar **desaprobada!!!**

# Herramientas de Test para la Evaluación

- En la tarea también les adjuntamos **herramientas de testing** de JUnit
- Estas herramientas las utilizamos en la cátedra como parte de la **evaluación** de los proyectos
- En el adjunto van a encontrar una **guía** de como agregarlas al proyecto (está armada respecto a **IntelliJ IDEA**)

**Aclaración Importante:** Las herramientas **no incluyen** los **casos de prueba** necesarios para **verificar adecuadamente** el compilador. El desarrollo de los casos es **responsabilidad del estudiante**. *Ademas, estas no son las únicas herramientas que utiliza la cátedra para la evaluación*

# Informe

# Informe

- El informe deberá indicar **como compilar el código fuente** entregado y como correrlo una vez compilado.
- Debe presentarse el diagrama de clases de los ASTs y una breve descripción de que se controla en cada tipo de nodo.
- Incluir los **tipos de errores semánticos** que el analizador puede de detectar.
- Deberá incluirse la **documentación** usual, propia de un **proyecto de software** (decisiones de diseño, clases utilizadas, etc).
- Debe indicarse que **logros** se intentan alcanzar en la Etapa



# Logros Disponibles para la Etapa 4

# Logros Etapa 4



## Imbatibilidad Semantica II

*(no valido para la reentrega, ni etapas subsiguiente)*

El software entregado pasa correctamente toda la batería de prueba utilizada por la cátedra



## Entrega Anticipada Semántica I

*(no valido para la reentrega, ni etapas subsiguiente)*

La entrega debe realizarse 48hs antes de la fecha limite

# Logros Etapa 4

Requiere:



## Genericidad Controlada E4

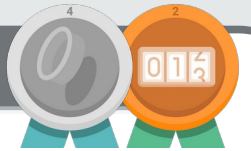
El compilador controla el correcto uso de clases genéricas en las sentencias y expresiones del programa. Los controles son como en Java, teniendo en cuenta las restricciones de representación. Al igual que en Java debe tenerse en cuenta como se instanció el tipo paramétrico y como manejar la creación de clases de usando la notación <>

Pueden ver este [link](#) y este [otro](#) de la documentación de Java, para mas detalles de que se puede hacer



# Logros Etapa 4

Requiere:



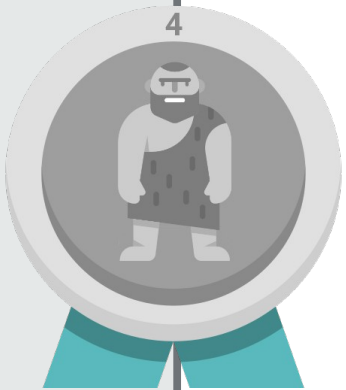
## For Each Generico E4

El compilador permite estructuras de repetición for each genericas y controla su uso correcto. A diferencia de Java en vez de requerir que la parte de la derecha conforme con `Iterable<T>`, en MiniJava se requiere que esa parte conforme con `Miniterable<T>`. La clase abstracta `Miniterable<T>` debe estar predefinida y tener la forma que se muestra abajo. El resto de los chequeos semánticos son como en Java.

```
abstract class MiniIterable<T>{  
    abstract void start();  
    abstract T next();  
    abstract boolean hasNext();  
}
```



# Logros Etapa 4

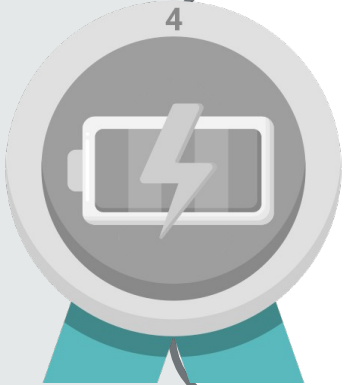


## Wrappers Primitivos E4

El compilador cuenta con las wrapper classes Integer, Character y Boolean. Estas clases funcionan como las homónimas de Java pero son mas simples. Solo tienen el constructor que recibe el valor primitivo que wrapean y solo proveen el método dinámico para obtener el valor primitivo que están wrapeando (intValue, charValue y booleanValue, respectivamente). Luego su comportamiento y uso es el mismo que en Java (donde, por ejemplo, pueden usarse como parte de expresiones).



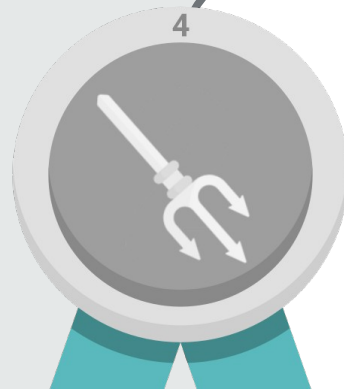
# Logros Etapa 4



## Sobrecarga E4

El compilador controla que se usen adecuadamente los métodos sobrecargados (y constructores si se alcanzo ese logro).

Requiere:



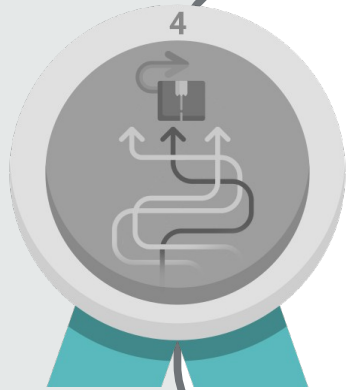
## Ternario E4

El compilador controla que las expresiones ternarias son correctamente tipadas

Requiere:

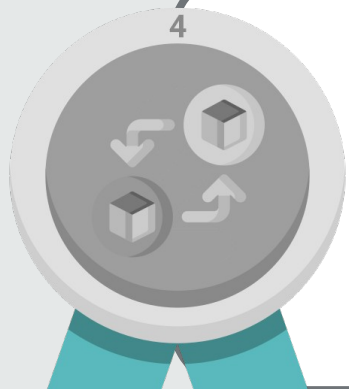


# Logros Etapa 4



## Todos los Caminos Llevan al Retorno

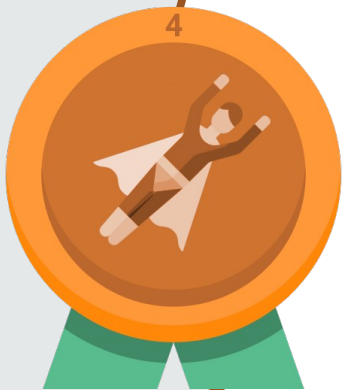
El compilador controla que cada camino de ejecución en el cuerpo de un método llegue a una sentencia de retorno (solo métodos con tipo de retorno no void). Si algún camino no lleva a return se reportará error.



## Coerción!

El compilador controla que se usen adecuadamente los literales de tipo Float y, además, permite coerción. En particular, permite coerción de ints a chars, chars a ints, chars a Strings y de ints a Strings. La coerción se aplica en los mismos contextos que en Java

# Logros Etapa 4



## Super! E4

El compilador permite utilizar **super** como en Java para acceder a métodos o atributos de un ancestro, o para invocar a su constructor

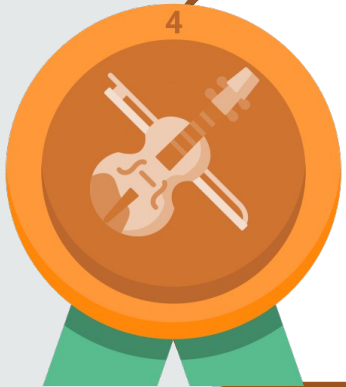


## Detección de Código Muerto

El compilador detecta las porciones inalcanzables de código (i.e., código en un bloque con posterioridad a una sentencia de retorno). Cuando lo detecta debe reportar un error.



# Logros Etapa 4

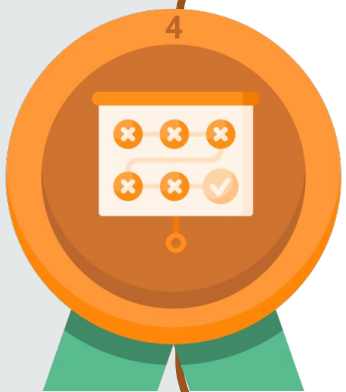


## Variables Locales Clásicas Controladas

Requiere:



El compilador controla las sentencias de definición de variables clásicas (en todas sus variantes) son correctamente tipadas



## Multi-Detección Errores Semanticos en Sentencias

El compilador no finaliza la ejecución ante el primer error semántico en una sentencia o expresión (se recupera) y es capaz de reportar otros errores que tenga el programa fuente en una corrida. Como mínimo se espera que el compilador pueda continuar con el análisis de la próxima sentencia

# Logros Etapa 4

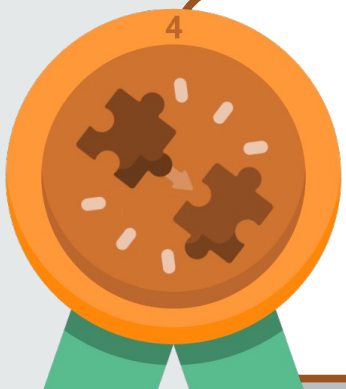


## Fors! Standard – Chequeado!

Requiere:

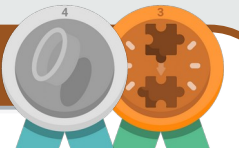


El compilador chequea que las sentencias fors standard (los que tienen 3 componentes) sean correctamente tipados. En particular, controla que la primer parte sea una asignacion, que la segunda sea una expresion booleana y que la tercera sea otra asignacion



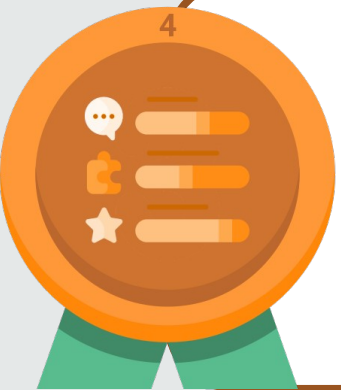
## Interfaces Genéricas E4

Requiere:



El compilador controla que las interfaces genéricas sean utilizadas adecuadamente en las sentencias del programa.

# Logros Etapa 4

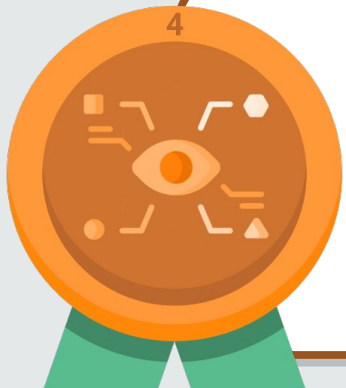


Requiere:

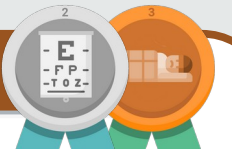


## Atributos Inicializados – Chequeado!

El compilador controla que los atributos inicializados son correctamente tipados (los controles son los mismos que en Java)



Requiere:



## Visibilidad - Chequeada

El compilador controla que los accesos sobre atributos, métodos y constructores respetan los modificadores de visibilidad. En particular, para los atributos tapados se comporta como en Java.

# Logros de Etapas Anteriores

- Recordar que es posible también intentar alcanzar **logros** de la **etapa anterior**
  - **Es importante** que nos aclaren como desactivar el analizador semántico si persiguen logros del sintáctico o el léxico
- Aun así chequear que el logro de sea alcanzable en una etapa posterior ya que existen algunos que no lo permiten (como por ejemplo entrega anticipada o imbatibilidad)